

Lựa Chọn Cấu Trúc Dữ Liệu Và Hiệu Quả Thuật Toán

Chen Hong

Trường Trung học trực thuộc Đại học Sư phạm Phúc Kiến

1999

Nguồn gốc: Data-structure choice and algorithmic efficiency, from IOI 1998 Picture

IOI China candidate team papers / 1999 / Chen Hong.pdf

Abstract

“Thuật toán + cấu trúc dữ liệu = chương trình”. Thiết kế thuật toán và chọn cấu trúc dữ liệu phù hợp là hai mặt bổ sung cho nhau trong lập trình. Bài viết lấy bài *Picture* của IOI 1998 làm ví dụ, so sánh cách dùng cấu trúc tuyến tính và cấu trúc cây, từ đó phân tích ảnh hưởng của lựa chọn cấu trúc dữ liệu tới hiệu quả thuật toán.

Từ khóa: lựa chọn cấu trúc dữ liệu; cấu trúc tuyến tính; cấu trúc cây.

1 Mở đầu

Thuật toán thường là yếu tố quyết định hiệu quả chương trình, nhưng mọi thuật toán cuối cùng đều phải được hiện thực trên một cấu trúc dữ liệu tương ứng. Tinh hoa của nhiều thuật toán chính là việc chọn được một cấu trúc dữ liệu phù hợp làm nền tảng. Vì vậy khi lập trình, không chỉ cần chú trọng thiết kế thuật toán, mà còn phải chọn đúng cấu trúc dữ liệu; lựa chọn tốt thường đem lại hiệu quả lớn.

Trong hai mặt thời gian và không gian, bài viết chủ yếu phân tích hiệu quả thời gian, tức độ phức tạp thời gian. Ta luôn mong chương trình cho kết quả mong muốn trong thời gian ngắn. Nếu tiết kiệm bộ nhớ quá mức khiến thời gian không thể chấp nhận thì việc giải bài cũng không có lợi.

2 Bài toán Picture và khung thuật toán

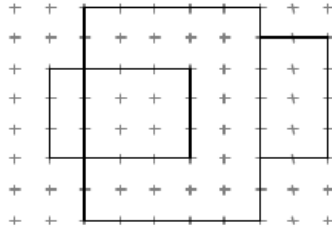
2.1 Phát biểu bài toán

Bài *Picture* của IOI 1998 được mô tả như sau. Trên tường dán một số hình chữ nhật như poster hoặc ảnh; mọi cạnh đều thẳng đứng hoặc nằm ngang. Một hình chữ nhật có thể bị các hình chữ nhật khác che một phần hoặc toàn bộ. Chu vi biên ngoài của hợp các hình chữ nhật được gọi là **chu vi đường bao**. Cần viết chương trình tính chu vi đường bao.

Giới hạn dữ liệu:

$$0 \leq \text{số hình chữ nhật} < 5000,$$

tọa độ là số nguyên trong đoạn $[-10000, 10000]$. Hình minh họa trong bản gốc có ba hình chữ nhật với chu vi đường bao bằng 30.



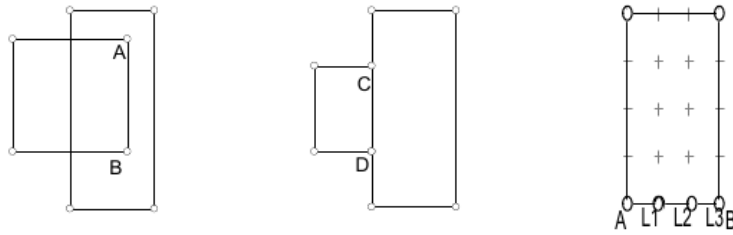
Hình 1: Ví dụ ba hình chữ nhật trong bài *Picture*; chu vi đường bao bằng 30.

2.2 Định nghĩa đường bao

Trước khi mô tả thuật toán, ta làm rõ khái niệm đường bao:

1. Đường bao gồm hữu hạn đoạn thẳng; mỗi đoạn là cạnh của một hình chữ nhật hoặc một phần của cạnh hình chữ nhật.
2. Đoạn thẳng tạo thành đường bao không được bị bất kỳ hình chữ nhật nào che phủ.

Bản gốc có hai hình nhỏ minh họa hai kiểu một đoạn cạnh bị che phủ.



Hình 2: Hai kiểu đoạn cạnh bị che phủ và cách chia cạnh AB thành các đoạn nguyên tố L_1, L_2, L_3 .

2.3 Đoạn nguyên tố

Một đặc điểm quan trọng của bài là ta phân tích các cạnh hình chữ nhật. Vì đầu mút cạnh, tức đỉnh hình chữ nhật, có tọa độ nguyên và phạm vi tọa độ bị giới hạn, ta có thể xem mặt phẳng như một lưới. Các cạnh hình chữ nhật chắc chắn nằm trên các đường lưới.

Với một đoạn thẳng trên lưới, điều ta quan tâm là tọa độ tuyệt đối của nó. Ví dụ, một cạnh AB có thể được chia thành các đoạn nhỏ L_1, L_2, L_3 nối hai đỉnh lưới kề nhau. Những đoạn cơ bản như vậy được gọi là **đoạn nguyên tố**. Đoạn nguyên tố là đơn vị cơ bản tạo nên cạnh hình chữ nhật và đường bao. Một đoạn nguyên tố hoặc hoàn toàn thuộc đường bao, hoặc hoàn toàn không thuộc đường bao.

Cách định nghĩa này rời rạc hóa bài toán, đưa việc nghiên cứu về từng đơn vị cơ bản. Tuy nhiên, tổng số đoạn nguyên tố trên toàn mặt phẳng là rất lớn:

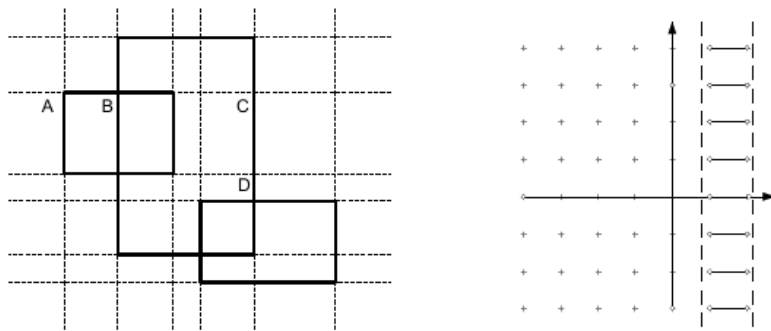
$$20001 \cdot 20000 \cdot 2.$$

Nếu phạm vi tọa độ là $[-10^6, 10^6]$, số đoạn nguyên tố sẽ lên tới $8 \cdot 10^{12}$. Vì vậy cần tối ưu hóa khái niệm này.

2.4 Siêu đoạn nguyên tố

Ta định nghĩa một đơn vị cải tiến là **siêu đoạn nguyên tố**. Nó thu được từ việc cắt mặt phẳng bằng các đường thẳng đi qua cạnh của các hình chữ nhật. Cụ thể, với N hình chữ nhật, dùng hoành độ của $2N$ cạnh dọc để cắt mặt phẳng theo phương dọc, và dùng tung độ của $2N$ cạnh ngang để cắt theo phương ngang. Mặt phẳng bị chia thành $(2N + 1)^2$ miền.

Các đoạn như một cạnh ngang AB giữa hai đường cắt dọc liền kề, hoặc cạnh dọc CD giữa hai đường cắt ngang liền kề, là siêu đoạn nguyên tố. Chúng có tính chất tương tự đoạn nguyên tố: là đơn vị cơ bản của đường bao. Khác biệt là số lượng siêu đoạn nguyên tố ít hơn nhiều, khoảng $4N$ đoạn theo các cạnh liên quan, và không phụ thuộc vào độ lớn miền tọa độ.



Hình 3: Cắt mặt phẳng theo các cạnh hình chữ nhật và nhóm các siêu đoạn nguyên tố nằm giữa hai đường cắt kề nhau.

Vì ưu điểm đó, thuật toán sẽ nghiên cứu các siêu đoạn nguyên tố.

2.5 Rời rạc hóa và khung thuật toán

Ta không xét từng siêu đoạn nguyên tố riêng lẻ, vì vẫn có thể tốn thời gian; cũng không xét toàn bộ cùng lúc, vì khi đó khó xử lý. Một cách trung gian là chia các siêu đoạn nguyên tố thành nhóm. Các siêu đoạn kẹp giữa hai đường cắt dọc tự nhiên tạo thành một nhóm: chúng có cùng độ dài và cùng hoành độ hai đầu. Các đoạn dọc cũng xử lý tương tự.

Sau rời rạc hóa, khung thuật toán cơ bản là:

1. Cắt mặt phẳng bằng các tọa độ cạnh hình chữ nhật.
2. Đặt bộ cộng $ans \leftarrow 0$.
3. Xét từng nhóm siêu đoạn nguyên tố, kiểm tra trong nhóm đó phần nào thuộc đường bao, rồi cộng độ dài tương ứng vào ans .
4. In ans .

Khung này còn thô; phần tinh chỉnh phụ thuộc vào cấu trúc dữ liệu được chọn.

3 Lựa chọn thứ nhất: cấu trúc tuyến tính

3.1 Xây dựng ánh xạ

Nền tảng thuật toán là rời rạc hóa mặt phẳng. Để cắt mặt phẳng, ta cần ghi lại các tọa độ cắt. Cách trực quan là dùng mảng một chiều: chỉ số biểu diễn số thứ tự điểm cắt, phần tử mảng biểu diễn tọa độ. Với mảng, việc truy cập thuận tiện và có thể sắp xếp trong $\mathcal{O}(N \log N)$.

Một kiểu ánh xạ có thể mô tả như sau:

Len	số điểm cắt,
Coord[1..Max]	tọa độ các điểm cắt.

Các thao tác gồm:

- Create: đặt $Len \leftarrow 0$;
- Insert(X): tăng Len , rồi ghi X vào $Coord[Len]$;
- Sort: sắp xếp các tọa độ.

Ví dụ với hoành độ, ta khởi tạo X_map , chèn hoành độ của mọi cạnh dọc hình chữ nhật, rồi sắp xếp. Tung độ xử lý tương tự bằng Y_map . Phần này đáp ứng tốt yêu cầu thuật toán: Create và Insert là $\mathcal{O}(1)$, còn Sort là $\mathcal{O}(N \log N)$ và chỉ chạy một lần.

3.2 Mảng tuyến tính cho một nhóm siêu đoạn

Sau khi xây dựng ánh xạ, mặt phẳng đã được cắt. Vấn đề chính là mô tả trạng thái của một nhóm siêu đoạn nguyên tố. Vì cần tính chu vi đường bao, ta quan tâm trong một nhóm có bao nhiêu siêu đoạn thuộc đường bao. Xét các nhóm đoạn ngang. Giả sử:

- các nhóm siêu đoạn được đánh số $1, \dots, 2N - 1$;
- nhóm s có độ dài $Length(s)$;
- nhóm s có $Belong(s)$ siêu đoạn thuộc đường bao.

Khi đó tổng độ dài cạnh ngang của đường bao là

$$ANS_h = \sum_{s=1}^{2N-1} Length(s) \cdot Belong(s).$$

$Length(s)$ dễ tính bằng hiệu hai tọa độ cắt kề nhau; vì vậy chỉ cần tính $Belong(s)$.

Sau rời rạc hóa, số siêu đoạn trong một nhóm ngang hữu hạn, xấp xỉ $2N$. Điều này gợi ý dùng mảng một chiều:

$$A[1..MaxSize].$$

3.3 Quét cộng dồn

Ta xem mỗi phần tử $A[i]$ là một bộ đếm:

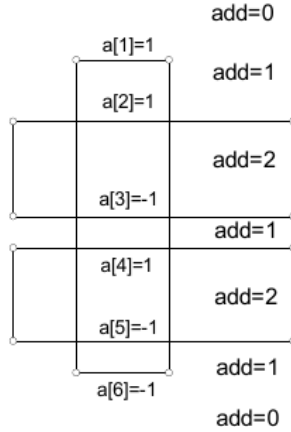
$$A[i] = \#\{\text{cạnh trên phủ siêu đoạn } i\} \\ - \#\{\text{cạnh dưới phủ siêu đoạn } i\}.$$

Sau đó dùng một biến add , quét từ trên xuống dưới và cộng dồn $A[i]$. Một siêu đoạn i thuộc đường bao trong hai trường hợp:

1. $A[i] \neq 0$ và trước khi cộng $A[i]$, $add = 0$. Khi đó siêu đoạn là cạnh trên.
2. $A[i] \neq 0$ và sau khi cộng $A[i]$, $add = 0$. Khi đó siêu đoạn là cạnh dưới.

Với một nhóm siêu đoạn, quá trình tính $Belong(s)$ gồm hai phần:

1. gán giá trị cho $A[i]$, tức bước tích lũy;



Hình 4: Quét cộng dồn trên một nhóm siêu đoạn nguyên tố: các giá trị $A[i]$ làm thay đổi trạng thái phủ add .

2. quét từ trên xuống dưới, duy trì add , tức bước quét.

Giải mã rút gọn:

```

COUNT( $s$ ) :
1       $A \leftarrow 0$ 
2      for  $I = 1$  to  $N$ 
3          if rectangle  $I$  vượt qua nhóm  $s$ 
4               $A[\text{cạnh trên của } I] += 1$ 
5               $A[\text{cạnh dưới của } I] -= 1$ 

ADDING( $s$ ) :
1      COUNT( $s$ )
2       $add \leftarrow 0, \text{ sum} \leftarrow 0$ 
3      for  $i = 1$  to chỉ số tung độ lớn nhất
4          if  $A[i] \neq 0$ 
5              if  $add = 0$  then  $\text{sum} += 1$ 
6               $add += A[i]$ 
7              if  $add = 0$  then  $\text{sum} += 1$ 
8      return  $\text{sum}$ .

```

Mỗi lần ADDING tốn $\mathcal{O}(N)$. Có $2N - 1$ nhóm siêu đoạn ngang, nên tính chu vi ngang tốn $\mathcal{O}(N^2)$. Chu vi dọc tương tự, vì vậy toàn bộ bài *Picture* theo cấu trúc tuyến tính có độ phức tạp $\mathcal{O}(N^2)$. Đây là đa thức, nhưng khi N gần 5000, lượng tính toán vẫn lớn.

4 Phân tích thêm về lựa chọn cấu trúc dữ liệu

Quá trình quét cộng dồn thể hiện một cách nhận thức vấn đề, và dùng mảng một chiều làm nền. Nhưng liệu có cách tốt hơn không? Khi chọn cấu trúc dữ liệu cần xét vài mặt sau.

Phù hợp với mô tả trạng thái. Cấu trúc dữ liệu phải mô tả được trạng thái của bài toán và cho phép định nghĩa rõ các thao tác trên trạng thái. Với *Picture*, trạng thái là một nhóm siêu đoạn nguyên tố, mục tiêu là xác định số siêu đoạn thuộc đường bao. Mảng tuyến tính mô tả trực quan và dễ cài đặt, nhưng không hiệu quả vì mỗi nhóm được quét độc lập; kết quả nhóm sau không dùng được kết quả nhóm trước.

Phù hợp với thuật toán. Cấu trúc dữ liệu phục vụ thuật toán, nên phải xét các thao tác của thuật toán. Ngược lại, cấu trúc dữ liệu cũng ảnh hưởng đến thiết kế thuật toán. Chẳng hạn, nếu cần heap sort trên một dãy, nên chọn cấu trúc định vị nhanh như mảng, chứ không nên chọn danh sách liên kết. Trong bài này, rời rạc hóa cần ghi điểm cắt, nên mảng ánh xạ là lựa chọn tự nhiên.

Ảnh hưởng của cấu trúc dữ liệu tới thuật toán thể hiện ở hai mặt. Thứ nhất là năng lực lưu trữ: cấu trúc lưu nhiều thông tin giúp thuật toán dễ thiết kế hơn, nhưng thường tốn bộ nhớ hơn. Thứ hai là các thao tác được định nghĩa trên cấu trúc: thao tác giống như công cụ; công cụ tốt giúp thiết kế thuật toán dễ hơn. Mảng tuyến tính có thao tác đơn giản, nhưng không liên kết tốt thống kê giữa các nhóm siêu đoạn khác nhau.

Thuận tiện lập trình. Nhiều cấu trúc phức tạp cho hiệu quả cao nhưng khó cài đặt và dễ sai. Khi đó, nếu có thể chọn cấu trúc quen thuộc mà không giảm hiệu quả quá nhiều thì đó là một thỏa hiệp hợp lý. Trong bài này, để tìm chỉ số ánh xạ của một cạnh hình chữ nhật, nếu xây dựng ánh xạ ngược từ tọa độ về chỉ số sẽ phức tạp hơn. Khi độ phức tạp chính vẫn là $\mathcal{O}(N)$ cho mỗi lần quét, việc tiếp tục tối ưu ánh xạ không thay đổi bản chất hiệu quả, nhưng làm tăng khó khăn cài đặt.

Linh hoạt dùng kiến thức đã có. Nhiều vấn đề mới có liên hệ nội tại với vấn đề cũ. Cấu trúc dữ liệu “mới” đôi khi là sự kết hợp hữu cơ của các cấu trúc cơ bản, hoặc được gọi ra từ cấu trúc cơ bản. Nếu tìm được một cấu trúc kinh điển phù hợp như hàng đợi, ngăn xếp, danh sách liên kết, cây, v.v., việc hiện thực thường nhẹ hơn và đáng tin cậy hơn.

Tóm lại, hướng rời rạc hóa của bài *Picture* là đúng, vì dữ liệu có tới gần 5000 hình chữ nhật và cần giảm quy mô. Hiệu quả chưa cao chủ yếu do lựa chọn cấu trúc tuyến tính. Nút thắt nằm ở việc thống kê các nhóm siêu đoạn tách rời nhau. Để khắc phục, ta thử dùng cấu trúc cây, cụ thể là cây đoạn cân bằng, gọi là **cây siêu đoạn nguyên tố**; thực chất chính là cây đoạn.

5 Lựa chọn thứ hai: cấu trúc cây

5.1 Cây đoạn

Với một nhóm siêu đoạn, số đoạn thuộc đường bao liên quan chặt chẽ đến vị trí các cạnh dọc của hình chữ nhật cắt qua nhóm đó. Chiều các cạnh dọc lên trục Y , ta được các khoảng đóng Q . Dùng cây đoạn để mô tả hợp các khoảng đóng này.

Cây đoạn là một cây nhị phân cân bằng mô tả một hoặc nhiều khoảng. Ta cần biết trước các giá trị có thể có của đầu mút khoảng. Giả sử

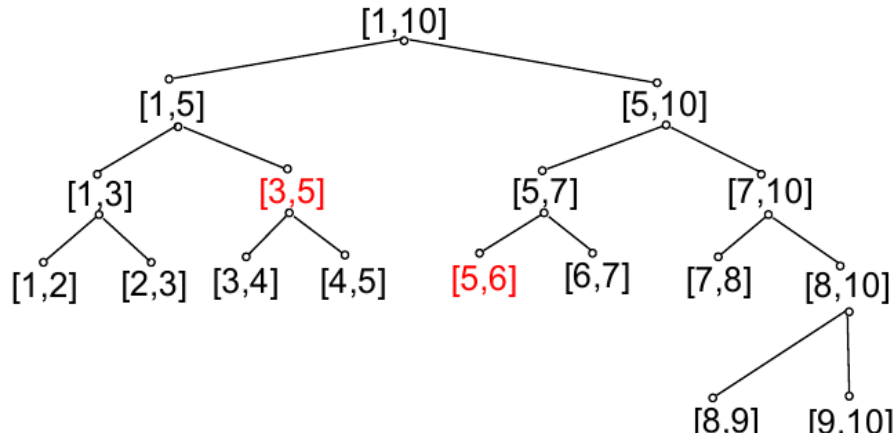
$$a[1] < a[2] < \dots < a[N]$$

là tập đầu mút đã sắp xếp. Với mọi khoảng cần mô tả $[x, y]$, tồn tại $1 \leq i \leq j \leq N$ sao cho $x = a[i]$ và $y = a[j]$. Trong cây đoạn, các chỉ số i, j mới là ý nghĩa chính.

Cây đoạn chia trục số thành các khoảng sơ cấp $[i, i + 1]$ với $i = 1, \dots, N - 1$. Mỗi khoảng sơ cấp ứng với một lá. Mỗi nút trong ứng với một khoảng $[i, j]$ với $j - i > 1$. Một nút có các trường:

i, j	chỉ số hai đầu khoảng,
count	số khoảng phủ toàn bộ nút này,
leftchild, rightchild	hai con.

Thủ tục xây cây BUILD(l, r) đặt $i = l, j = r, \text{count} = 0$. Nếu $r - l > 1$, chia tại $k = \lfloor (l + r)/2 \rfloor$, xây con trái $[l, k]$ và con phải $[k, r]$; nếu không, nút là lá. Cây có chiều cao $\lceil \log N \rceil$ và xây trong $\mathcal{O}(N)$.



Hình 5: Một cây đoạn với các đầu nút $1, \dots, 10$; các nút màu đỏ biểu diễn khoảng $[3, 6]$ bằng hợp $[3, 5] \cup [5, 6]$.

5.2 Chèn và xóa khoảng

Cây đoạn hỗ trợ chèn khoảng $\text{INSERT}(l, r)$:

```

1  if  $l \leq a[i]$  and  $a[j] \leq r$ 
2    count += 1
3  else
4    if  $l < a[(i + j) \div 2]$  then chèn vào con trái
5    if  $r > a[(i + j) \div 2]$  then chèn vào con phải.
```

Xóa khoảng $\text{DELETE}(l, r)$ tương tự, chỉ thay tăng count bằng giảm count. Điều kiện tiên quyết là khoảng $[l, r]$ đã được chèn và chưa bị xóa. Cả chèn và xóa đều có độ phức tạp $\mathcal{O}(\log N)$.

5.3 Độ đo

Vì cây đoạn được định nghĩa đệ quy, độ đo của hợp khoảng cũng có thể duy trì đệ quy. Thêm trường M , là độ đo của hợp khoảng trong cây con nút hiện tại:

$$M = \begin{cases} a[j] - a[i], & \text{count} > 0, \\ 0, & \text{nút là lá và count} = 0, \\ M_{\text{trái}} + M_{\text{phải}}, & \text{nút trong và count} = 0. \end{cases}$$

Sau mỗi lần chèn hoặc xóa, gọi UPDATE để cập nhật M . Tại một nút, cập nhật mất $\mathcal{O}(1)$, nên tổng độ phức tạp của chèn/xóa vẫn là $\mathcal{O}(\log N)$.

5.4 Số đoạn liên tục

Ở đây số đoạn liên tục nghĩa là hợp các khoảng có thể phân giải thành bao nhiêu khoảng độc lập. Ví dụ

$$[1, 2] \cup [2, 3] \cup [5, 6]$$

tạo thành hai đoạn liên tục: $[1, 3]$ và $[5, 6]$.

Thêm các trường:

line, lbd, rbd.

$lbd = 1$ nếu đầu trái i được phủ, ngược lại bằng 0; rbd định nghĩa tương tự cho đầu phải j . Khi $count > 0$, cả hai đều bằng 1. Nếu là lá và $count = 0$, cả hai bằng 0. Nếu là nút trong và $count = 0$, thì

$$lbd = \text{left.lbd}, \quad rbd = \text{right.rbd}.$$

Số đoạn liên tục:

$$\text{line} = \begin{cases} 1, & \text{count} > 0, \\ 0, & \text{lá và count} = 0, \\ \text{left.line} + \text{right.line} - 1, & \text{nút trong, count} = 0, \\ \text{left.rbd} = 1, \text{right.lbd} = 1, \\ \text{left.line} + \text{right.line}, & \text{nút trong, count} = 0 \text{ và hai biên không nối nhau.} \end{cases}$$

Trong cài đặt có thể viết gọn:

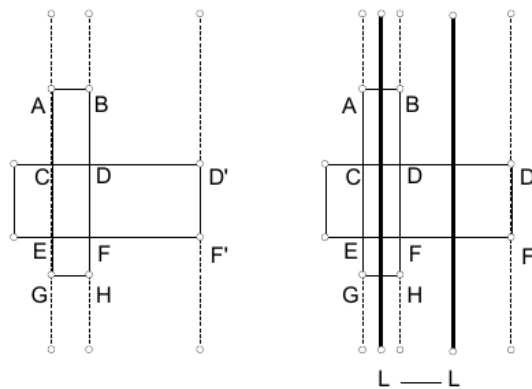
$$\text{line} = \text{left.line} + \text{right.line} - \text{left.rbd} \cdot \text{right.lbd}.$$

Như vậy cây đoạn đầy đủ có các trường $i, j, \text{count}, \text{line}, \text{lbd}, \text{rbd}, M$ và hai con, cùng các thao tác BUILD, INSERT, DELETE, và cập nhật.

6 Quét bằng cây đoạn cho Picture

Cấu trúc tuyến tính kém hiệu quả vì các nhóm siêu đoạn không liên hệ với nhau. Thực tế, các nhóm kề nhau có liên hệ rất rõ. Khi quét từ trái sang phải, một hình chữ nhật bắt đầu tại cạnh trái sẽ che một đoạn theo trục Y ; khi gặp cạnh phải, đoạn che phủ ấy biến mất. Đây chính là trạng thái có thể duy trì động bằng cây đoạn.

Đặt một đường quét L đi từ trái sang phải và dừng tại mỗi nhóm siêu đoạn. Trạng thái của L được biểu diễn bằng cây đoạn; mỗi cạnh dọc của hình chữ nhật là một khoảng cần hợp nhất. Số đoạn liên tục của cây đoạn nhân 2 cho biết số siêu đoạn ngang thuộc đường bao trong nhóm đó. Ví dụ trong hình gốc, nếu trạng thái là $[G, A] \cup [E, C]$, số đoạn liên tục là 1, nên số đoạn đường bao ngang của nhóm là 2.



Hình 6: Hai trạng thái kề nhau của đường quét: khi đoạn che phủ biến mất, các đoạn DD' và FF' trở thành một phần của đường bao.

Khi quét:

- trạng thái ban đầu rỗng, tức cây đoạn vừa xây xong;
- gặp cạnh trái hình chữ nhật thì chen khoảng tung độ của nó;

- gặp cạnh phải hình chữ nhật thì xóa khoảng tung độ của nó.

Vì quét từ trái sang phải, trước khi xóa cạnh phải, cạnh trái tương ứng đã được chèn, nên thao tác xóa hợp lệ.

Quá trình quét còn tính được chu vi dọc. Trước khi trạng thái thay đổi, cây có độ đo M_1 ; sau thay đổi có độ đo M_2 . Phần cạnh dọc mới lộ ra hoặc bị che đi có độ dài

$$|M_2 - M_1|.$$

Các lần chèn/xóa được gọi là **sự kiện**; đoạn cần chèn/xóa là **đoạn sự kiện**. Trước khi quét, sắp xếp sự kiện theo hoành độ.

Thuật toán quét bằng cây đoạn:

```

1  Rời rạc hóa bằng tọa độ đỉnh hình chữ nhật, lập  $X\_Map, Y\_Map$ .
2  Sắp xếp các sự kiện theo hoành độ.
3   $ROOT.BUILD(1, 2N)$ .
4   $lastM \leftarrow 0, \quad lastLine \leftarrow 0, \quad ans \leftarrow 0$ .
5  for each event  $e$  in order
6    if  $e$  là cạnh trái then  $Insert(e.y_1, e.y_2)$ 
7    else  $Delete(e.y_1, e.y_2)$ 
8     $nowM \leftarrow ROOT.M, \quad nowLine \leftarrow ROOT.line$ 
9     $ans += 2 \cdot lastLine \cdot (x_e - x_{prev})$ 
10    $ans += |nowM - lastM|$ 
11    $lastM \leftarrow nowM, \quad lastLine \leftarrow nowLine$ .
```

Trong bản trích xuất PDF, dòng công thức khoảng cách ở bước 9 có nhầm ký hiệu Y_Map ; theo nội dung thuật toán, khoảng cách đúng là hiệu hai hoành độ sự kiện liên tiếp.

Sắp xếp sự kiện tốn $\mathcal{O}(N \log N)$. Có $2N$ sự kiện, mỗi chèn/xóa tốn $\mathcal{O}(\log N)$, nên toàn bộ quá trình có độ phức tạp $\mathcal{O}(N \log N)$.

7 So sánh hai cách hiện thực

Hai cấu trúc dữ liệu khác nhau không chỉ khác về cách lưu trữ và thao tác, mà còn tạo ra độ phức tạp thuật toán khác nhau. Cấu trúc tuyến tính dẫn tới $\mathcal{O}(N^2)$, còn cấu trúc cây dẫn tới $\mathcal{O}(N \log N)$.

Bản gốc đưa một nhóm dữ liệu thử trên Celeron 300 với Borland Pascal. Với vài bộ nhỏ, cả hai đều dưới 1 giây. Với bộ lớn hơn, cách tuyến tính khoảng 30 giây, trong khi cách dùng cây vẫn dưới 1 giây. Điều này cho thấy khác biệt hiệu quả là rất rõ.

8 Mở rộng

Từ phân tích đặc trưng của bài *Picture* và cách dùng cấu trúc cây, có thể mở rộng bài toán.

Trước hết, tư tưởng rời rạc hóa cho phép tọa độ đỉnh từ số nguyên mở rộng sang số thực. Trong thuật toán và cấu trúc dữ liệu đã chọn, ta không còn dùng tính chất đặc biệt của số nguyên. Để thích ứng, chỉ cần đổi kiểu cơ sở của $Mapped.coord$, trường M của cây đoạn, và các bộ cộng như ans sang kiểu số thực.

Quan trọng hơn, bài toán chu vi cũng có thể mở rộng thành bài toán diện tích. Chu vi dùng số đoạn liên tục và độ đo của đường quét: phần ngang liên quan tới $line$, phần dọc liên quan tới M . Diện tích thì chỉ cần độ đo M . Trong quá trình quét, nếu sự kiện hiện tại có hoành độ x_i , diện tích dải vừa quét qua là

$$M \cdot (x_i - x_{i-1}).$$

9 Kết luận

Với bài *Picture*, trên nền tư tưởng rời rạc hóa, việc đổi cấu trúc dữ liệu từ tuyến tính sang cây đã làm giảm mạnh độ phức tạp thời gian. Sự cải tiến bắt nguồn từ lựa chọn cấu trúc dữ liệu, và sâu hơn là từ việc hiểu rõ hơn bài toán cũng như đặc điểm của cấu trúc dữ liệu.

Thảo luận trên cho thấy tầm quan trọng của việc chọn cấu trúc dữ liệu. Điểm quan trọng nhất là cấu trúc dữ liệu phải phù hợp với bài toán và phù hợp với thuật toán; chỉ khi đó nó mới mô tả trạng thái một cách bản chất và giúp giải quyết vấn đề.

Ở một ý nghĩa nào đó, cấu trúc dữ liệu và thuật toán là biểu hiện cụ thể của không gian và thời gian. Chúng có lúc thống nhất, có lúc mâu thuẫn. Trong *Picture*, cấu trúc tuyến tính không dùng nhiều bộ nhớ động nhưng có độ phức tạp $\mathcal{O}(N^2)$; cấu trúc cây dùng nhiều bộ nhớ hơn nhưng giảm thời gian xuống $\mathcal{O}(N \log N)$. Điều hòa mâu thuẫn này và đưa chúng về thống nhất là một khâu quan trọng của thiết kế thuật toán.

Tài liệu tham khảo

- Fu Qingxiang và Wang Xiaodong, *Thuật toán và cấu trúc dữ liệu*, Nhà xuất bản Công nghiệp Điện tử.
- Niklaus Wirth, *Thuật toán + cấu trúc dữ liệu = chương trình*, bản dịch của Cao Dehe và Liu Chunlian, Nhà xuất bản Khoa học.